

Software Engineering

Software Architecture

What is the difference between System and Software Architecture?

System architecture encompasses the overall design and organisation of a system, which includes both hardware and software components.

It defines the structure and behavior of the entire system, including the software, hardware, networks, data storage, and other critical elements.

Software architecture refers to the high-level structure of a software system and the discipline of creating such structures.

It defines how the software components or modules are organised and how they interact with each other.

What is Software Architecture?

Software Architecture

Software Architecture is heavily influenced by the **programming language** that you chose.

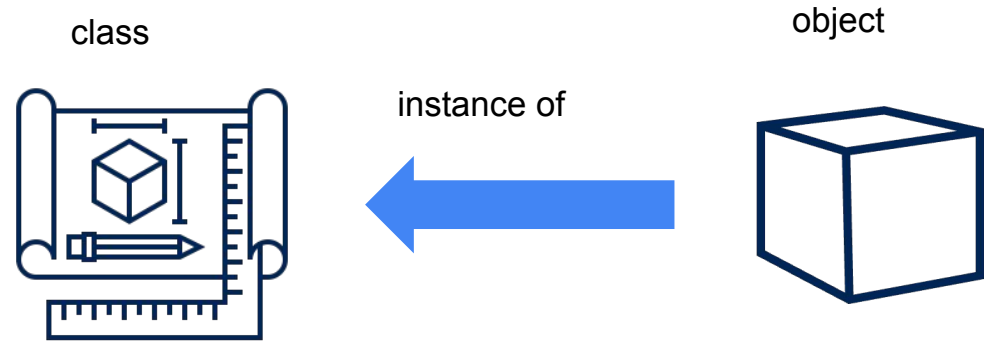
Most influential language paradigms are

- Procedural programming
- **Object Oriented Programming (OOP)**
- Functional Programming (FP)

Object Oriented Programming (OOP)

Objects are
instances of classes

Classes are
blueprints for objects



Object Oriented Programming

- Why is OOP helpful?
 - Facilitates modularisation and therefore maintainability due to **separation of concerns**
 - Enables parallelisation by breaking down the work into separate parts and therefore working with multiple Software Engineers or teams (**divide and conquer**)
 - Standard **patterns** help to solve standard problems
- Languages
 - Java, Ruby, VB.NET, C#, C++ and more

Object Oriented Paradigms

 **Encapsulation:** Group functionality in to meaningful bundles

 **Inheritance:** Reusability of functionality

 **Polymorphism:** Variants of reused functionality

 **Abstraction:** Definition of abstract base that can be fleshed out by specific functions

Encapsulation

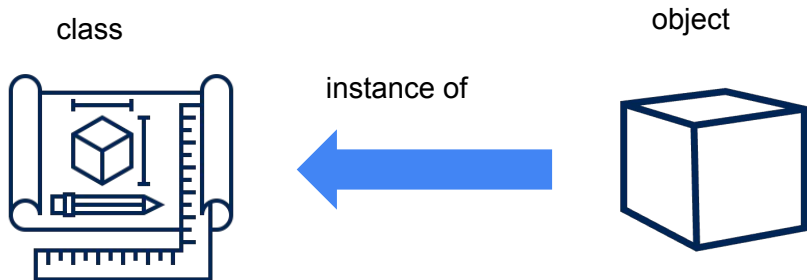
A language construct that enables

- **Bundling**

- Data is packaged meaningfully as members (aka variables)
- Methods are introduced to access and manipulate that data

- **Information hiding**

- Access to members and methods can be restricted



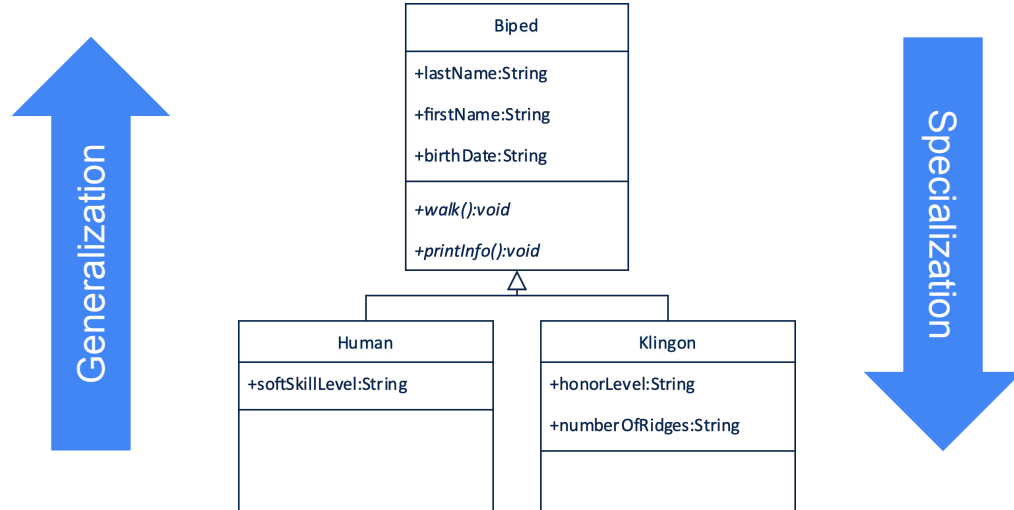
<i>Student</i>
+majorSubject:String
+minorSubject:String
+courseList:List<Course>
+joinCourse(c:Course):void
+dropCourse(c: Course):void

<i>Marcus:Student</i>
+majorSubject = "Business Informatics"
+minorSubject = "Commerce"
+courseList = {AOIS, SE}

<i>Daniela:Student</i>
+majorSubject = "Mathematics"
+minorSubject = "Commerce"
+courseList = {LALGI, SE}

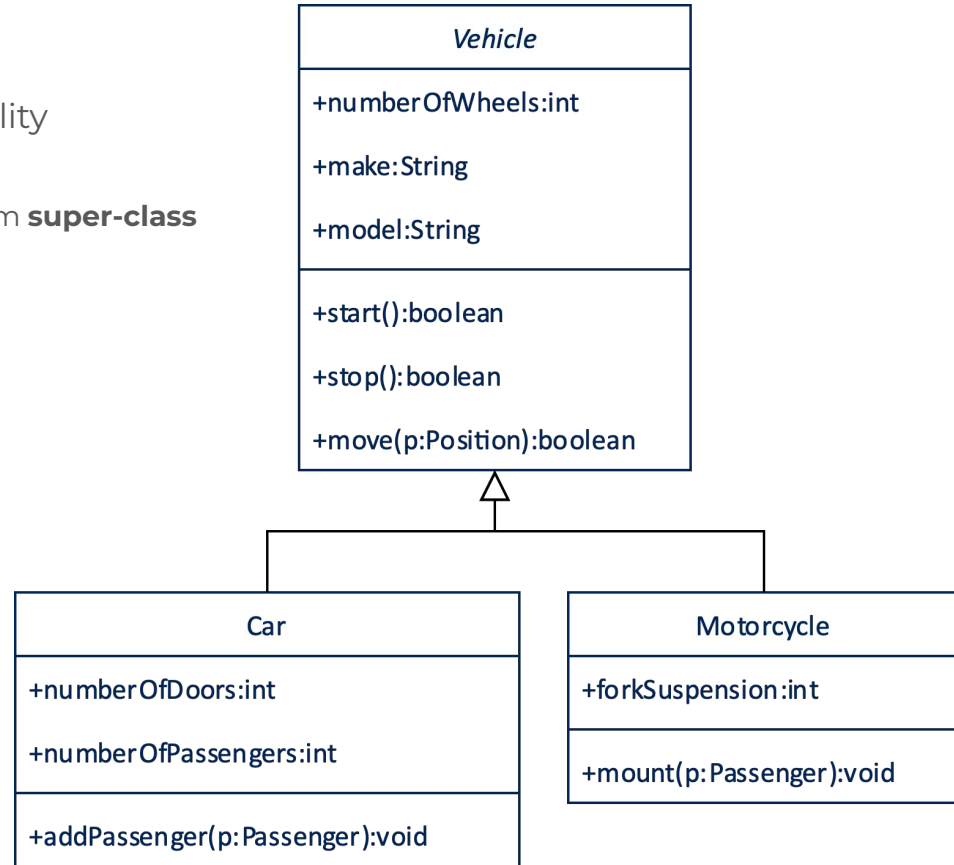
Inheritance

- Generalization consolidates common or shared attributes and behaviours
 - Use generalization to reduce or eliminate redundancies
- Specialization incorporates attributes and behaviours unique to the object
 - Use specialization to build specific functionality



Inheritance

- Enables reuse of data structures and functionality
 - Is set up as a **class hierarchy**
 - **sub-classes** inherit data structures and methods from **super-class**
- Example
 - Car **is a** vehicle, Motorcycle **is a** vehicle
 - Vehicle **has** attributes (make, model, etc.)
 - Car and motorcycle inherit these
 - Vehicle **has** methods (start, stop, etc.)
 - Car and motorcycle inherit these methods
 - Car **has** special attributes (doors).
 - These are not shared with vehicle or motorcycle
 - Motorcycle **has** special attributes (fork)
 - - These are not shared with vehicle or car
- The inheritance direction is unidirectional
 - Every *Car* is a *Vehicle* - But not every *Vehicle* is a *Car*
 - Assigning a *Car* to a *Vehicle* is therefore possible
 - Whereas assigning a *Vehicle* to a *Car* is not



Polymorphism

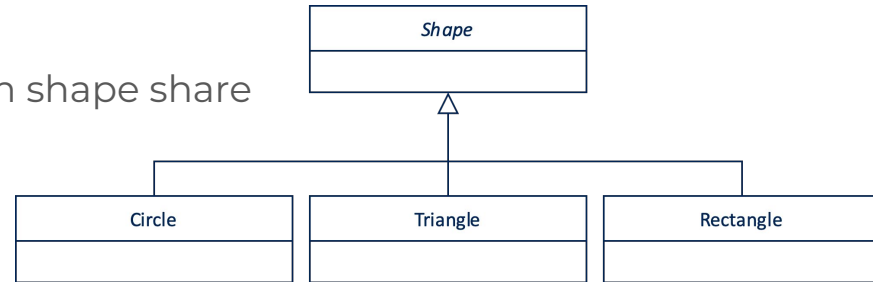
- Builds on top of inheritance through
 - modification of functionality
 - **overriding** methods of a super-class
- Methods
 - Executable code that manipulates state
 - Always states a **return type**
 - **void** if no return type
 - Can have a list of **parameters**
 - Followed by a block of statements
 - name and parameters make up the **method signature**

```
modifier returnType name(param) {  
    statement 1  
    .  
    .  
    .  
    statement N  
}
```

For a method to be overridden by its sub-class **name**, **return type** (can be sub-type) and **parameters** have to be identical to its parent!

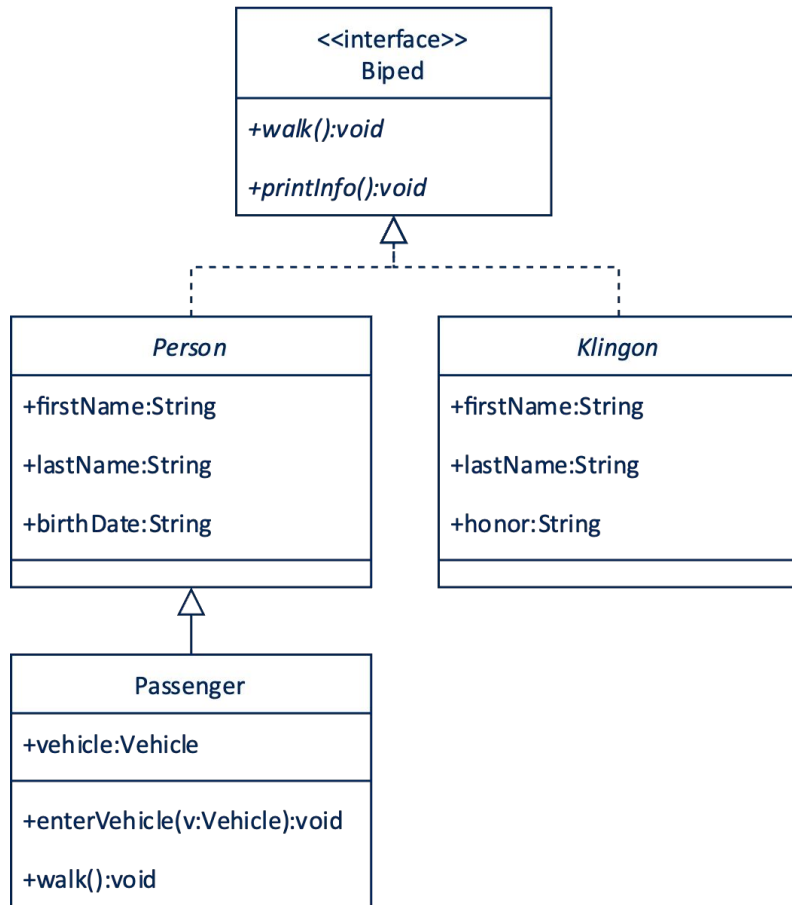
Abstraction

- An abstract structure can
 - implement common features (structure, members and methods)
 - Pre-define (to be implemented) required features
- while not being instantiable -
 - meaning that, we can not create an object out of an abstract class.
- Example: Geometric shape
 - A generic shape can not exist
 - Yet it defines general features, that all other concrete shapes inheriting from shape share



Abstraction

- In some languages we can also use **interfaces**
 - Interfaces are contracts to which an implementing class adheres
 - Interface do not define members
 - They can define constants
- We create interfaces with the keyword **interface**
- Classes that adhere to an interface mention it by stating the keyword **implements** after the class name followed by the interface name
- Classes can implement multiple interfaces



Recap: Object Oriented Paradigms

 **Encapsulation:** Group functionality in to meaningful bundles

 **Inheritance:** Reusability of functionality

 **Polymorphism:** Variants of reused functionality

 **Abstraction:** Definition of abstract base that can be fleshed out by specific functions

Live Coding: Demonstration of Object Oriented Paradigms

Help me find the speed that is helpful to you

- Ask questions
- Make suggestions
- Say if I should slow down or repeat

Learning to code

Coding schools

- <https://code.org/> - large variety of learning programmes
- <https://processing.org/> - Visual Framework to learn basic Java
Syntax before learning OOP

Techniques to analyse code

- Tracing / Printing
- Tests (later)
- Debugger (later)

Learning to code

AI can act as a pairing partner / teacher

- Be careful not just to copy paste - try to understand the code you use
- Turn off / ignore AI from time to time for half an hour and code without AI to check your learning progress

AI Tools

- <https://chatgpt.com/>
- <https://www.phind.com/> - focuses on coding

AI Agents

- <https://www.anthropic.com/> - Claude with Sonnet
- <https://jules.google/>
- <https://www.cursor.com/> - focuses on coding, integrated in IDE
- <https://www.jetbrains.com/help/idea/ai-assistant-in-jetbrains-ides.html> - IDE IntelliJ with AI
- <https://code.visualstudio.com/docs/copilot/overview> - IDE VSCode with AI
- <https://lmstudio.ai/> - locally installed

Software Architecture Patterns

Software Architecture: Design Patterns

- Creational

- Builder
- Abstract Factory
- Prototype
- **Singleton**

Decoupling of
object creation

- Structural

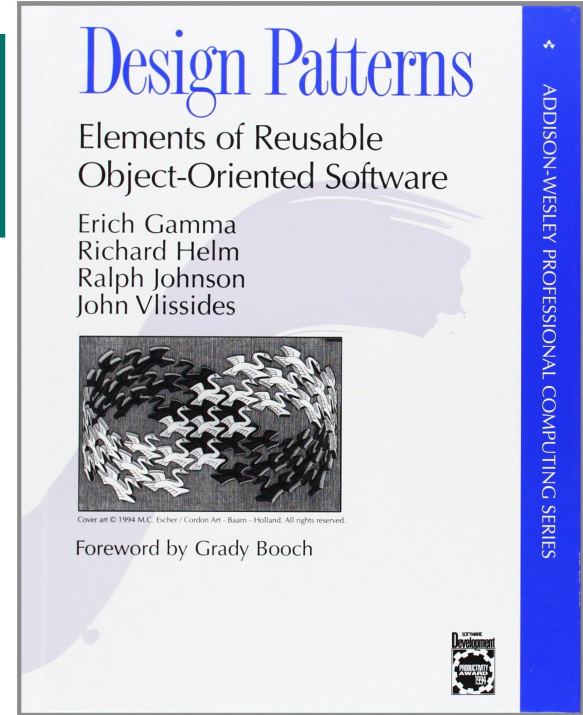
- **Adapter**
- Bridge
- Composite
- Decorator
- **Facade**
- Flyweight
- Proxy

Object
composition

- Behavioural

- Chaining
- Command
- Interpreter
- Iterator
- **Mediator**
- Memento
- **Observer**
- State
- **Strategy**
- Template
- Visitor

Communication
between objects

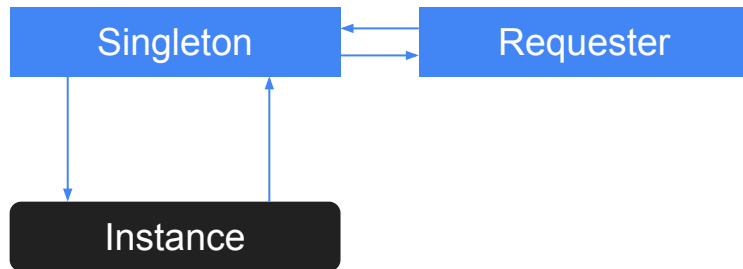


Design Patterns by Erich Gamma et al.

Creational: Singleton

- Restricts creation of objects to a single instance
- Use sparsely, introduces global state

```
class Singleton {  
    private static Singleton instance;  
    ...  
    public static Singleton getInstance() {  
        synchronized (instance) {  
            if(instance == null) {  
                instance = new Singleton();  
            }  
        }  
        return instance;  
    }  
}
```

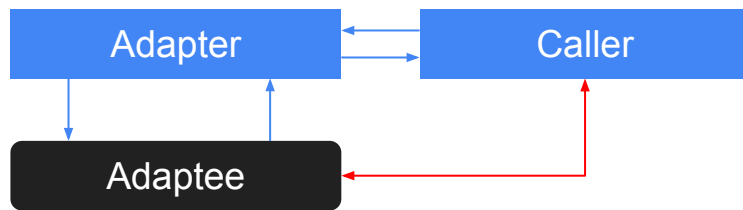


Structural: Adapter

- Can be used to adapt / retrofit incompatible types
- Acts as a wrapper around the adaptee

```
public static void main(String[] args) {  
    USBCharger usbCharger = new USBCharger();  
    iPhone iphone = new iPhone();  
    Android android = new Android();  
    LightningToUSBAdapter adapter = new LightningToUSBAdapter(iphone);  
  
    usbCharger.charge(android);  
    usbCharger.charge(iphone);  
    usbCharger.charge(adapter);  
}
```

Android charging
Cannot connect lightning phone to usb
LightningToUSBAdapter adapted iPhone
iPhone charging



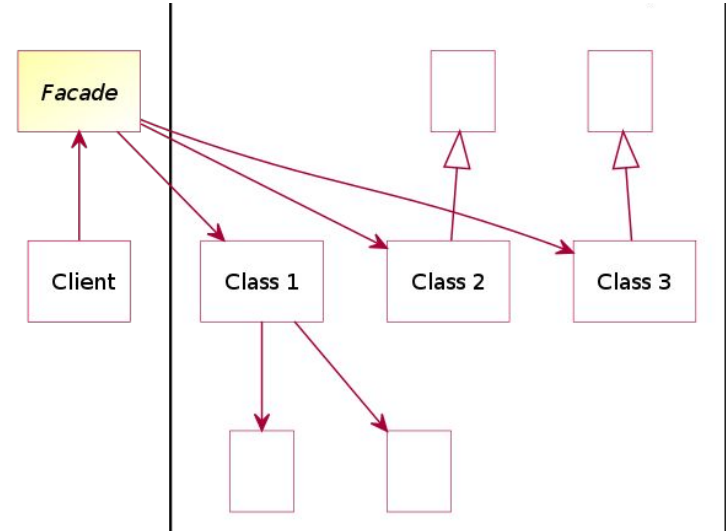
Structural: Facade

- Masks more complex underlying structure

```
public static void main(String[] args) {
    FacadePhone phone = new FacadePhone();
    phone.turnOn();
}
```

```
class FacadePhone{
    Powersupply powersupply = new Powersupply();
    Display display = new Display();
    Modem modem = new Modem();
    CPU cpu = new CPU();
    GPU gpu = new GPU();
```

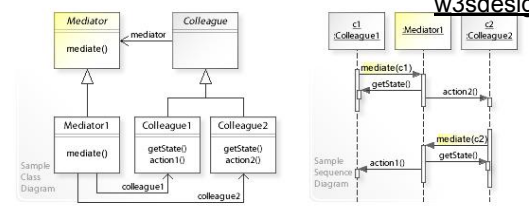
```
void turnOn() {
    powersupply.turnOn();
    cpu.turnOn();
    gpu.turnOn();
    display.turnOn();
    modem.turnOn();
    System.out.println("=> Phone turnedOn");
}
```



```
class Display{void turnOn() {System.out.println("Display turned on");}}
class Modem{void turnOn() {System.out.println("Modem turned on");}}
class CPU{void turnOn() {System.out.println("CPU turned on");}}
class GPU{void turnOn() {System.out.println("GPU turned on");}}
class Powersupply{void turnOn() {System.out.println("Powersupply turned on");}}
```

Behavioural: Mediator

- Decouples objects by encapsulating the interaction logic



```

public static void main(String[] args) {
    Mediator gradeOffice = new GradeOffice();
    Lecturer may = new Lecturer("MsMay", gradeOffice);
    Student robert = new Student("Robert", "1234", gradeOffice);
    may.gradeStudentForCourse("10", "1234", "SE");
}
  
```

➡ Robert received 10 points for SE!

Behavioural: Observer

- Notifies all registered observers of posted events
- Also known as publish / subscribe (PubSub)

```
public static void main(String[] args) {  
    Chat messageBoard = new Chat();  
    Chatter juls = new Chatter("Julian");  
    Chatter lisa = new Chatter("Lisa");
```

```
juls.participate(messageBoard);
```

```
lisa.participate(messageBoard);
```

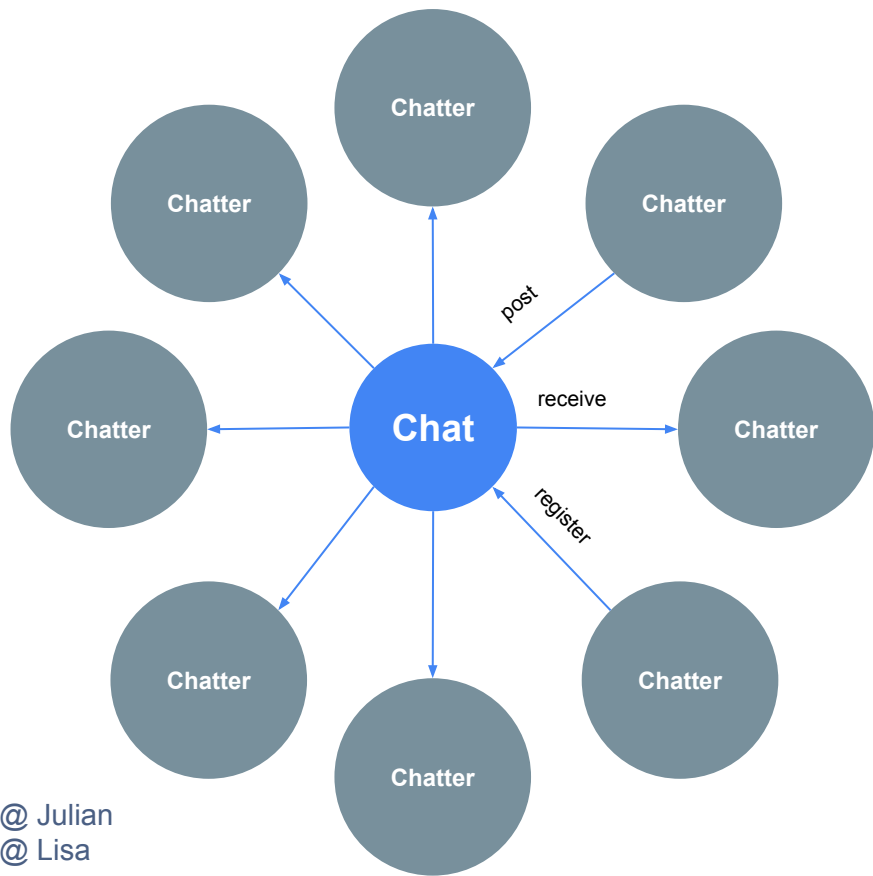
```
juls.postMessage("Hi Lisa!");
```

```
lisa.postMessage("Hi Juls!");
```

From Chat to participants:

From Julian: Hi Lisa! received @ Julian
From Julian: Hi Lisa! received @ Lisa

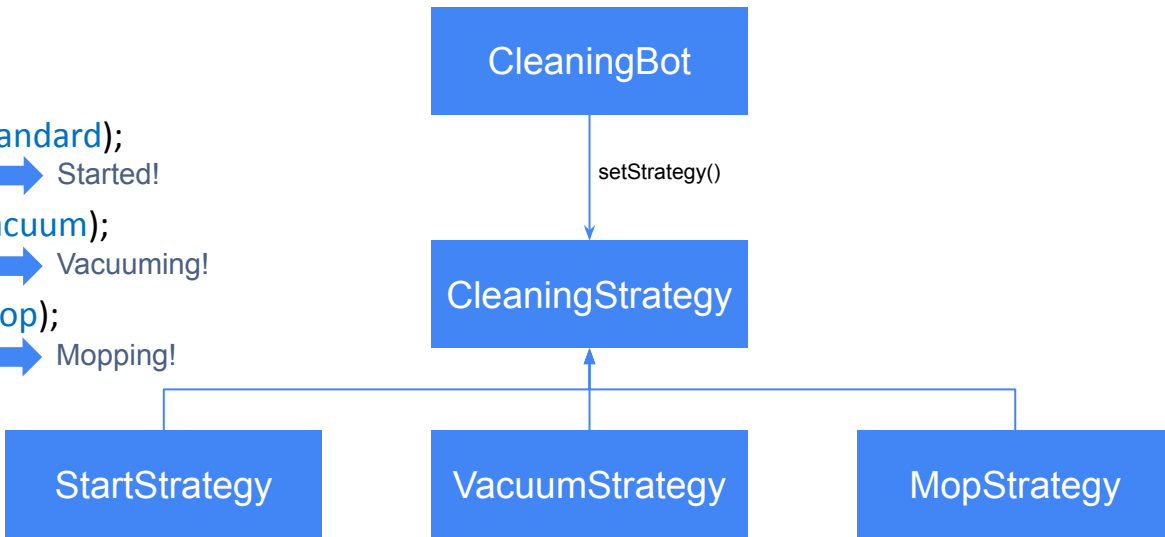
From Lisa: Hi Juls! received @ Julian
From Lisa: Hi Juls! received @ Lisa



Behavioural: Strategy

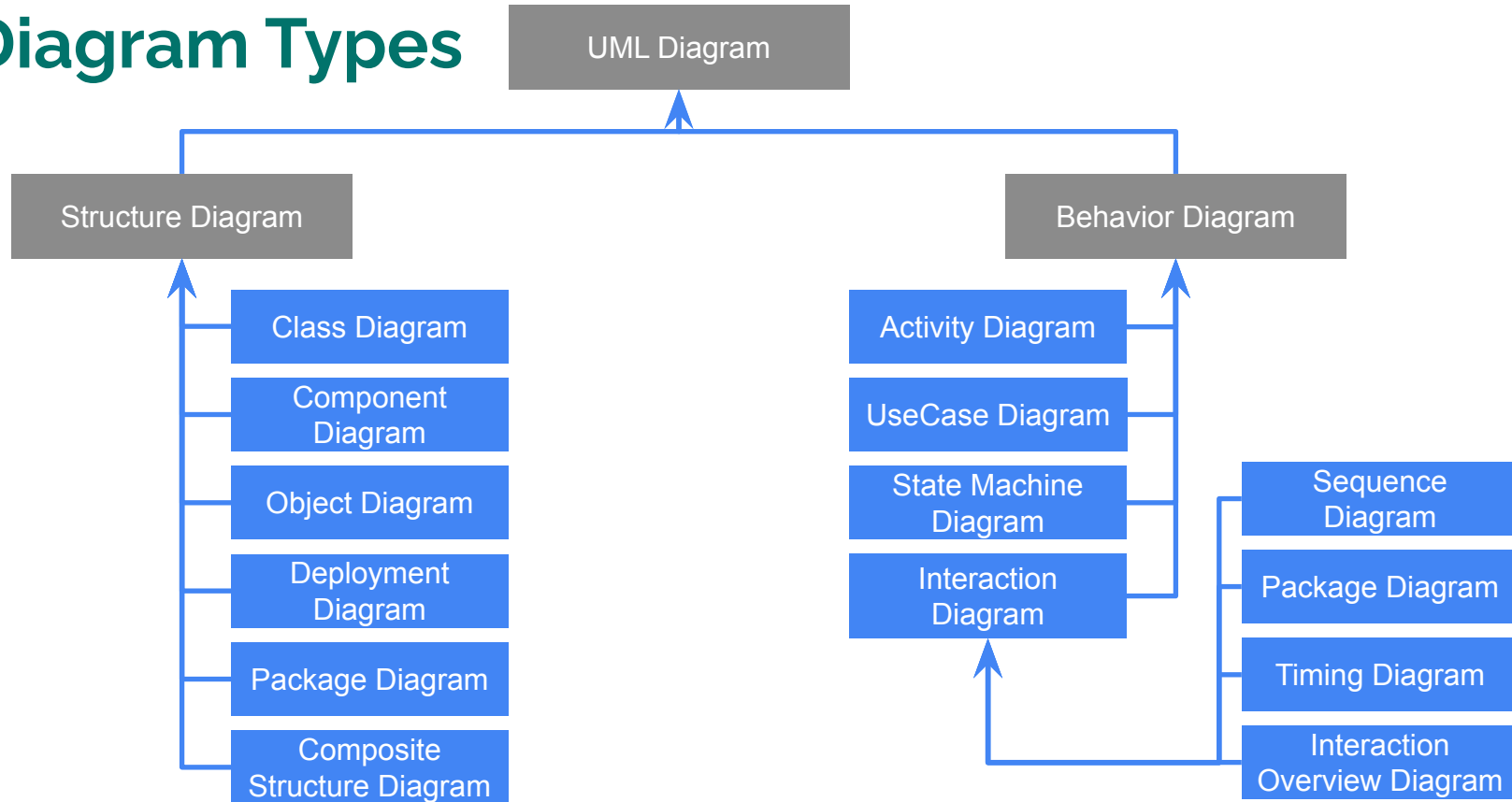
- Object changes its behaviour based selected strategy
- Enables changing of algorithm at runtime

```
public static void main(String[] args) {  
    CleaningBot bot = new CleaningBot();  
    bot.setStrategy(CleaningStrategy.Type.standard);  
    bot.doWork();           ➡ Started!  
    bot.setStrategy(CleaningStrategy.Type.vacuum);  
    bot.doWork();           ➡ Vacuuming!  
    bot.setStrategy(CleaningStrategy.Type.mop);  
    bot.doWork();           ➡ Mopping!  
}
```



How to document architecture?

UML Diagram Types



Arc42: a framework to structure architecture

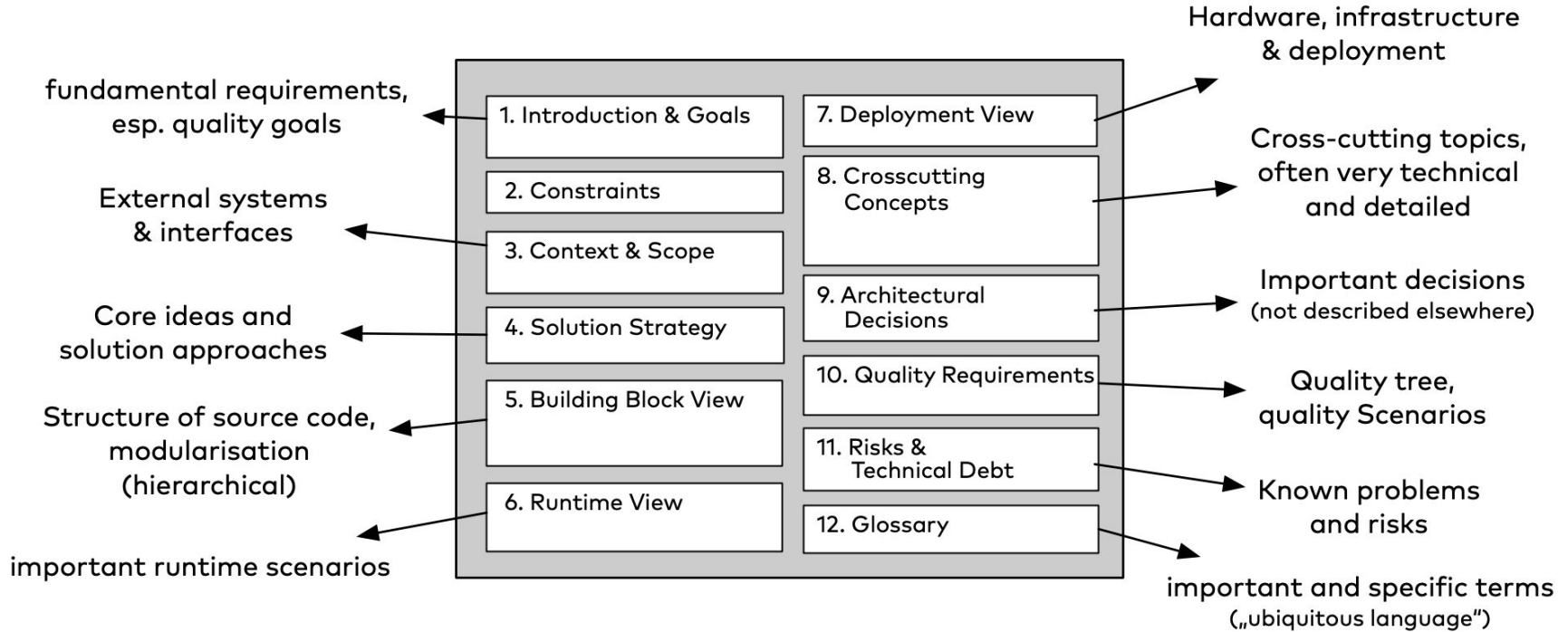


Image source: <https://arc42.org/overview>

Architecture Decision Records

[Template by Michael Nygard:](#)

Title: These documents have names that are short noun phrases. For example, "ADR 1: Deployment on Ruby on Rails 3.0.10" or "ADR 9: LDAP for Multitenant Integration"

Context: This section describes the forces at play, including technological, political, social, and project local. These forces are probably in tension, and should be called out as such. The language in this section is value-neutral. It is simply describing facts.

Decision: This section describes our response to these forces. It is stated in full sentences, with active voice. "We will ..."

Status: A decision may be "proposed" if the project stakeholders haven't agreed with it yet, or "accepted" once it is agreed. If a later ADR changes or reverses a decision, it may be marked as "deprecated" or "superseded" with a reference to its replacement.

Consequences: This section describes the resulting context, after applying the decision. All consequences should be listed here, not just the "positive" ones. A particular decision may have positive, negative, and neutral consequences, but all of them affect the team and project in the future.

Architecture Decision Records

ADRs facilitate

- onboarding of new team members
- making decisions based on facts not feelings
- revisiting and adjusting decisions if constraints change
- making decisions clear so that everyone can comply with them

To use their full potential they best are

- a regular practise in a Software Engineering team
- a mixture of synchronous and asynchronous decision making process
- stored close to the code so that they are discoverable

Exercise - 15 minutes

In groups of 2-3, create an ADR for the following scenario:

Your team decides to change the authentication and use Google and Facebook Login instead of the custom code and storing passwords yourself. Marketing has estimated that this would enable about 90% of your possible users to use this feature. Create an ADR for this architecture decision.

Title: Short noun phrase.

Status: Such as proposed, accepted, rejected, deprecated, superseded, etc.?

Context: What is the issue that is motivating this decision or change?

Decision: What is the change that we're proposing and/or doing?

Consequences: What becomes easier or more difficult to do because of this change?

What is the economic impact of software architecture (aka design)?

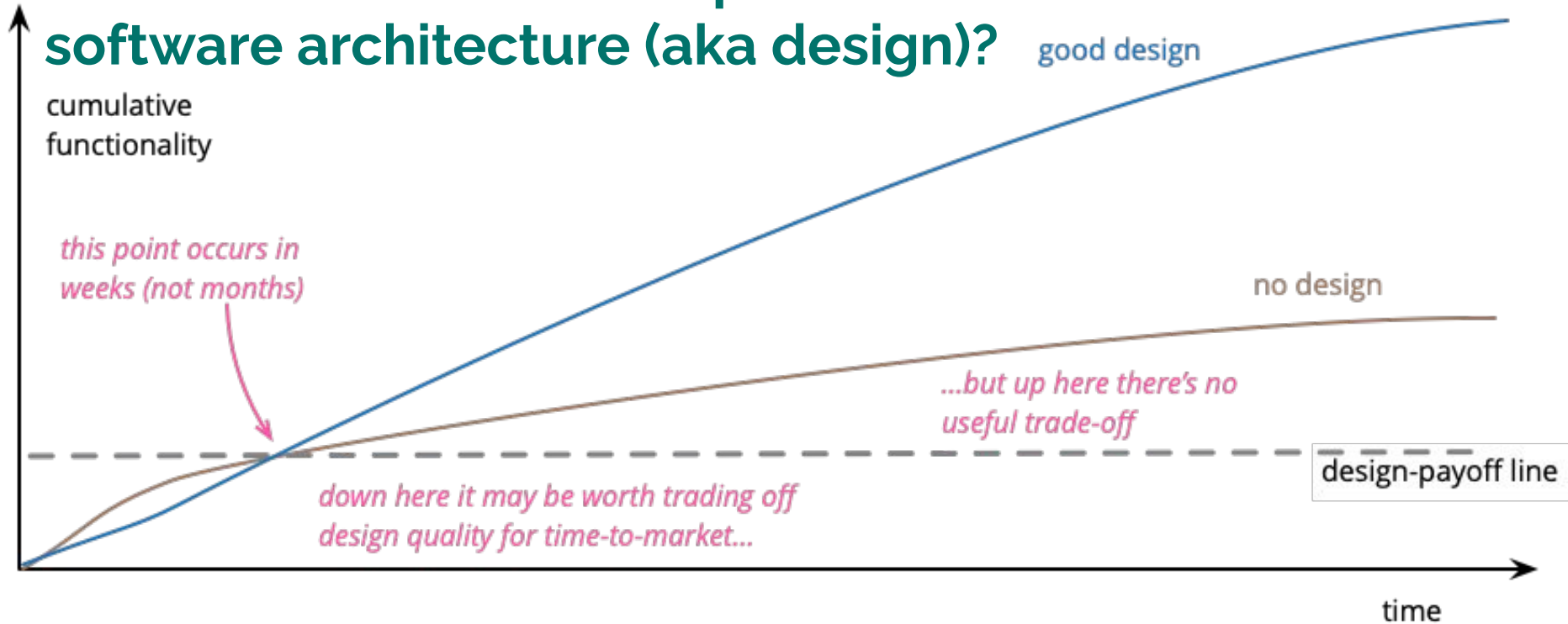
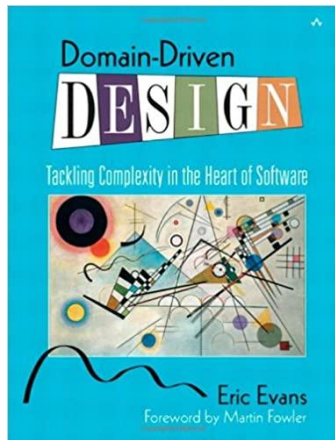


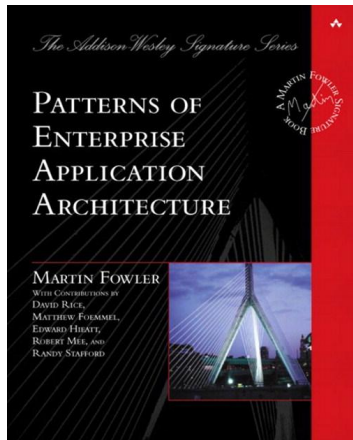
Image source: <https://martinfowler.com/bliki/DesignStaminaHypothesis.html>

Further Reading: System and Software Architecture

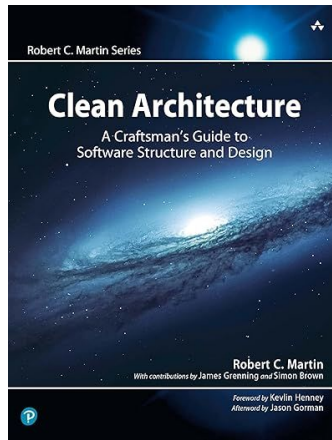
- New architectural patterns evolve all the time
- Learn the basics and you'll be able to adapt to new ones
- Architectural patterns can be described using component diagrams



Eric Evans



Martin Fowler



Robert C. Martin



Bill Wilder

